

## Projeto CNN

Bloco DotProduct: Responsável por fazer multiplicação e soma entre dois vetores.

Se chama Dot Product (produto interno) pela natureza da operação de mesmo nome: dados dois vetores quaisquer  $[x_1, x_2, x_3]$  e  $[y_1, y_2, y_3]$ , o produto interno entre eles é igual a  $(x_1 * y_1 + x_2 * y_2 + x_3 * y_3)$ . É o que o bloco implementa.

```
//gera os multiplicadores que fazem multiplicação elemento a elemento entre o vetor de entrada e os pesos
always @(posedge clk) begin
    for (m = 0; m < N_INPUTS; m = m + 1) begin
        mult[m] <= input_vec[m] * weight[m];
    end
end
```

Neste trecho, ele faz a multiplicação elemento a elemento entre o vetor de entrada e o vetor de pesos.

```
generate
for (s = 0; s < STAGES; s = s + 1) begin :gen_estagios

    //cada estágio executa N/2 vezes (N = número de elementos no estágio anterior), sendo N somado de 1, se teve resto no estágio anterior
    for (n = 0; n < ((N_INPUTS + (1 << s) - 1) >> s) >> 1; n = n + 1) begin :gen_elementos_soma
        always @(posedge clk) begin
            //primeiro estágio do pipeline de árvore de soma (soma dois do estágio anterior)
            if (s == 0) begin :primeiro_estagio
                soma[0][n] <= mult[2 * n] + mult[2 * n + 1];
            //próximos estágios do pipeline de árvore de soma (soma dois do estágio anterior)
            end else begin
                soma[s][n] <= soma[s - 1][2 * n] + soma[s - 1][2 * n + 1];
            end
        end
    end

    if (s == 0) begin :gen_contorno_primeiro_estagio
        always @(posedge clk) begin
            //persiste o valor do elemento ímpar (sozinho) para o próximo estágio
            if (N_INPUTS % 2 == 1) soma[0][N_INPUTS >> 1] <= mult[N_INPUTS - 1];
        end
    end else begin :gen_contorno_outros_estagios
        always @(posedge clk) begin
            //persiste o valor do elemento ímpar (sozinho) para o próximo estágio
            if (((N_INPUTS + (1 << (s - 1)) - 1) >> (s - 1)) % 2 == 1) begin
                soma[s][((N_INPUTS + (1 << s) - 1) >> s) >> 1] <= soma[s - 1][((N_INPUTS + (1 << s) - 1) >> s) - 1];
            end
        end
    end
end
endgenerate
```

Neste trecho, declara uma árvore de soma de tamanho apropriado (necessário testar se o tamanho da árvore é suficiente), com suficientes estágios de soma. O número de estágios e número de somadores por estágio depende do tamanho dos vetores, pois quanto maiores os vetores, mais somadores e estágios é necessário ter.

```

//pipeline de valid, o valid_in é enviado pelos blocos utilizadores deste bloco, para indicar que estão enviando uma entrada válida.
//depois de LATENCY ciclos de clock, baseados no número de estágios, a saída de um valid_in recebido se torna válida
always @(posedge clk) begin
    if (rst) begin
        valid_pipe <= '0;
    end else begin
        valid_pipe[0] <= valid_in;
        for (v = 1; v <= LATENCY; v = v + 1) begin
            valid_pipe[v] <= valid_pipe[v - 1];
        end
    end
end
end

assign out = soma[STAGES-1][0];
//saída válida corresponde ao último índice do pipeline de saída
assign valid_out = valid_pipe[LATENCY];

endmodule

```

Como a soma é feita por estágios, a saída não vai estar imediatamente pronta após um ciclo de clock. Na verdade, a árvore de soma é a maneira mais rápida de se obter uma saída válida. O bit indicador de saída válida é o valid\_out. Ele deve se tornar válido depois de LATENCY ciclos de clock, que são os ciclos suficientes para passar a soma por todos os estágios e chegar na saída.

```

module DotProduct #(
    parameter DATA_WIDTH = 8,
    parameter N_INPUTS = 32,

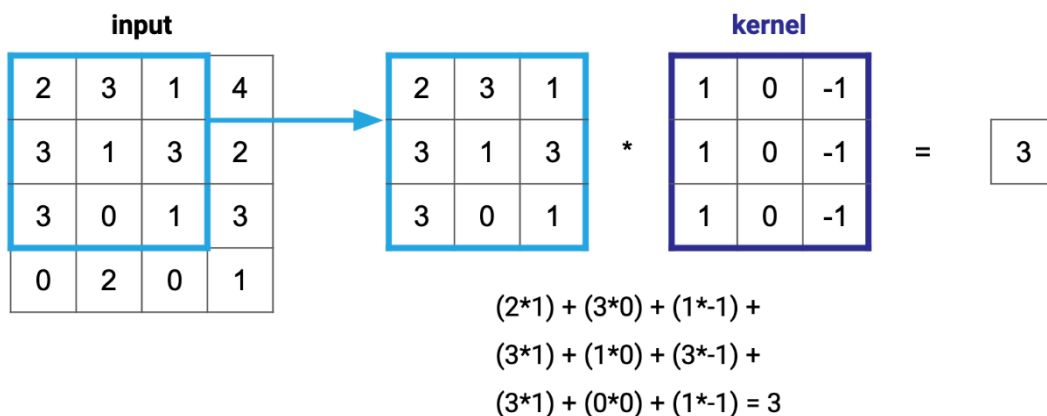
    localparam ACC_WIDTH = 2 * DATA_WIDTH + $clog2(N_INPUTS)
) (
    input logic clk,
    input logic rst,
    input logic signed [DATA_WIDTH-1:0] input_vec [0:N_INPUTS-1],
    input logic signed [DATA_WIDTH-1:0] weight [0:N_INPUTS-1],
    input logic valid_in,
    output logic valid_out,
    output logic signed [ACC_WIDTH-1:0] out
);

```

Em inputs, temos clk, rst, o vetor de entrada, o vetor de pesos, que serão multiplicados e seus resultados somados, o valid\_in, que determina quando chegou uma entrada válida para ele executar. Em outputs, temos o valid\_out, que determina quando há uma saída válida após os estágios de soma, e a própria saída, out.

Bloco Convolution: utiliza do DotProduct para fazer convolução entre um kernel e valores de uma imagem.

Neste caso, o vetor de entrada é uma janela da imagem, do mesmo tamanho do kernel, e o vetor de pesos é o próprio kernel. Cada peso do kernel se multiplica com seu correspondente valor do vetor de entrada, e após todas as multiplicações, os resultados se somam, para formar uma única saída.



A lógica do bloco permite que esse kernel deslize sobre a imagem, varrendo as linhas e colunas correspondentes e formando os valores de saída. Pode-se dizer que a saída é uma nova “imagem”, sendo mais comumente chamada de feature map, “mapa de características”.

```
module Convolution #(
    parameter DATA_WIDTH = 8,
    parameter IMG_SIZE = 5,
    parameter KERNEL_SIZE = 3,
    parameter PADDING = 0,

    localparam N_PIXELS = IMG_SIZE * IMG_SIZE,
    localparam N_KERNEL = KERNEL_SIZE * KERNEL_SIZE,
    localparam ACC_WIDTH = 2 * DATA_WIDTH + $clog2(N_KERNEL),
    localparam OUT_SIZE = IMG_SIZE + 2 * PADDING - KERNEL_SIZE + 1,
    localparam N_OUT = OUT_SIZE * OUT_SIZE,
    localparam BIT_COUNT = $clog2(N_OUT + 1),
    localparam BIT_CONV = $clog2(OUT_SIZE)
) (
    input logic clk,
    input logic rst,
    input logic signed [DATA_WIDTH-1:0] imagem [0:N_PIXELS-1],
    input logic signed [DATA_WIDTH-1:0] kernel [0:N_KERNEL-1],
    input logic start,
    output logic signed [ACC_WIDTH-1:0] result [0:N_OUT-1],
    output logic valid_out //Quando a convolução está concluída e a saída está pronta, valid_out se torna 1
);
```

Entradas: clk, rst, imagem, que é um vetor de pixels, kernel, que é um vetor de pesos em formato de janela, e start, que é um bit para ativação do bloco.

Saídas: result, que é o resultado da convolução, e valid\_out, seguindo a mesma lógica do bloco anterior.

Bloco FeatureMap: Declara N\_CHANNELS blocos de convolução. As imagens podem possuir mais de um canal, como os canais RGB. Portanto, para uma imagem RGB, são necessários 3 canais de convolução para extrair as características daquela imagem. Após a convolução dos kernels, os resultados são somados para formar a featureMap final.

```
integer i;
integer j;

always @(*) begin
    for (j = 0; j < N_OUT; j = j + 1) begin
        result[j] = {(ACC_WIDTH - DATA_WIDTH){bias[DATA_WIDTH-1]}, bias}; //estende o bias de DATA_WIDTH para ACC_WIDTH, respeitando o sinal (repete o último elemento na extensão)
        for (i = 0; i < N_CHANNELS; i = i + 1) begin
            result[j] += partial_result[i][j];
        end
    end
end

endmodule
```

Este bloco soma todos os resultados parciais dos kernels em um resultado final.

```
module FeatureMap #(
    parameter DATA_WIDTH = 8,
    parameter IMG_SIZE = 5,
    parameter KERNEL_SIZE = 3,
    parameter N_CHANNELS = 3,
    parameter PADDING = 0,

    localparam N_KERNEL = KERNEL_SIZE * KERNEL_SIZE,
    localparam ACC_WIDTH = 2 * DATA_WIDTH + $clog2(N_KERNEL),
    localparam ACC_WIDTH_FMAP = 2 * DATA_WIDTH + $clog2(N_KERNEL * N_CHANNELS),
    localparam OUT_SIZE = IMG_SIZE + 2 * PADDING - KERNEL_SIZE + 1,
    localparam N_OUT = OUT_SIZE * OUT_SIZE,
    localparam N_PIXELS = IMG_SIZE * IMG_SIZE
) (
    input logic clk,
    input logic rst,
    input logic start,
    input logic signed [DATA_WIDTH-1:0] imagem [0:N_CHANNELS-1][0:N_PIXELS-1],
    input logic signed [DATA_WIDTH-1:0] kernel [0:N_CHANNELS-1][0:N_KERNEL-1],
    input logic signed [DATA_WIDTH-1:0] bias,
    output logic signed [ACC_WIDTH_FMAP-1:0] result [0:N_OUT-1],
    output logic valid_out
);
```

Entradas: clk, rst, start, imagem, kernel, bias, que é um valor que se soma em conjunto com as multiplicações e somas sucessivas entre o kernel e a imagem (pense na função  $AX + B$ : A multiplica-se com X, e o B é um “offset” na função. O bias tem esse exato propósito.

Saídas: result (auto-explicativo) e valid\_out, que tem a mesma natureza dos demais.

Bloco ConvLayer: Declara N\_FILTERS FeatureMaps. Apenas uma feature map não extrai tudo que pode extrair de uma entrada. Para tanto, utilizamos vários filtros, para extrair diferentes características da entrada. Cada filtro terá N\_CHANNELS kernels, justamente para se adequar ao número de canais da entrada. Cada filtro gerará uma saída. As saídas de cada filtro não são somadas, mas representam os canais para a próxima camada. Por exemplo: Começo com uma imagem RGB (3 canais). Utilizo 5 filtros, cada um com 3 canais (para se adequar aos canais da entrada). Cada filtro gera uma saída (pois as saídas de cada canal se somam em uma), portanto fico com 5 saídas. Essas 5 saídas são, para a próxima camada, como uma imagem com 5 canais. A próxima camada deve, então, escolher filtros que tenham 5 canais para se adequar a essa nova entrada.

```
module ConvLayer #(
    parameter DATA_WIDTH = 8,
    parameter IMG_SIZE = 5,
    parameter KERNEL_SIZE = 3,
    parameter N_CHANNELS = 3,
    parameter N_FILTERS = 10,
    parameter PADDING = 0,

    localparam N_KERNEL = KERNEL_SIZE * KERNEL_SIZE,
    localparam OUT_SIZE = IMG_SIZE + 2 * PADDING - KERNEL_SIZE + 1,
    localparam N_OUT = OUT_SIZE * OUT_SIZE,
    localparam N_PIXELS = IMG_SIZE * IMG_SIZE,
    localparam ACC_WIDTH = 2 * DATA_WIDTH + $clog2(N_KERNEL * N_CHANNELS)
) (
    input logic clk,
    input logic rst,
    input logic start,
    input logic signed [DATA_WIDTH-1:0] imagem [0:N_CHANNELS-1][0:N_PIXELS-1],
    input logic signed [DATA_WIDTH-1:0] kernel [0:N_FILTERS-1][0:N_CHANNELS-1][0:N_KERNEL-1],
    input logic signed [DATA_WIDTH-1:0] bias [0:N_FILTERS-1],
    output logic signed [ACC_WIDTH-1:0] result [0:N_FILTERS-1][0:N_OUT-1],
    output logic valid_out
);
```

Possui mesmas entradas e saídas do FeatureMap, apenas diferindo no parâmetro N\_FILTERS, para declaração de várias FeatureMaps.

Bloco Neuron: Um neurônio. Realiza multiplicação e soma entre entradas e pesos correspondentes. Exatamente o que faz um dot product. Portanto, declara dotProducts para fazer essa operação. Também possui bias, e se aproxima ainda mais da ideia de um somatório de  $(A_i X_i + B_i)$ .

Possui uma particularidade: a função de ativação. Ela modifica a saída para uma forma desejável. Pode restringir a saída entre 0 e 1, -1 e 1, pode assumir outros formatos de saída. No nosso caso, utilizamos a ReLU, que transforma valores negativos em 0, e mantém todos os positivos neles mesmos.

```
module Neuron #(
    parameter DATA_WIDTH = 8,
    parameter WEIGHTS = 2,

    localparam ACC_WIDTH = 2 * DATA_WIDTH + $clog2(WEIGHTS)
) (
    input logic clk,
    input logic valid_in,
    input logic rst,
    input logic signed [DATA_WIDTH-1:0] x [0:WEIGHTS-1],
    input logic signed [DATA_WIDTH-1:0] w [0:WEIGHTS-1],
    input logic signed [DATA_WIDTH-1:0] b,
    output logic signed [ACC_WIDTH-1:0] z,
    output logic valid_out
);
```

Entradas: clk, valid\_in, rst, x (entrada), w (pesos), b (bias).

Saídas: z (resultado após função de ativação) e valid\_out.

Bloco FCLayer: Fully Connected Layer, é uma camada de neurônios. Diz-se fully connected pois cada entrada se conecta com cada neurônio, e cada neurônio se conecta com cada neurônio da camada seguinte. O mesmo não acontece com vetores de entrada e kernels, onde os pesos são reaproveitados diversas vezes.

Bloco mais pesado, declara N\_NEURONS neurônios que farão parte da camada.

```
module FCLayer #(
    parameter DATA_WIDTH = 8,
    parameter N_NEURONS = 10,
    parameter WEIGHTS = 2,
    parameter SHIFT = 1,

    localparam ACC_WIDTH = 2 * DATA_WIDTH + $clog2(WEIGHTS)
) (
    input logic clk,
    input logic valid_in,
    input logic rst,
    input logic signed [DATA_WIDTH-1:0] x [0:WEIGHTS-1],
    input logic signed [DATA_WIDTH-1:0] w [0:N_NEURONS-1][0:WEIGHTS-1],
    input logic signed [DATA_WIDTH-1:0] b [0:N_NEURONS-1],
    output logic signed [DATA_WIDTH-1:0] z [0:N_NEURONS-1],
    output logic valid_out
);
```

Possui mesmas entradas e saídas dos neurônios, diferindo no parâmetro N\_NEURONS, que declara o número de neurônios.

Bloco MaxPooling: Pega 4 valores de pixels de uma imagem (ou featureMap) e extrai apenas o maior entre os 4. Serve para redução do tamanho de imagens, e portanto, do processamento total da CNN.

```
define MAX2(a,b) (((a) > (b)) ? (a) : (b))

/* MaxPooling pega o maior elemento dentre quatro elementos e joga na saída. Imagem deve ser NxN com N par */
module MaxPooling #(
    parameter DATA_WIDTH = 8,
    parameter SIDE = 4,

    localparam SIDE_POOLED = SIDE / 2,
    localparam N_PIXELS = SIDE * SIDE,
    localparam N_PIXELS_POOLED = SIDE_POOLED * SIDE_POOLED
) (
    input logic signed [DATA_WIDTH-1:0] imagem_in [0:N_PIXELS-1],
    output logic signed [DATA_WIDTH-1:0] imagem_out [0:N_PIXELS_POOLED-1]
);

genvar i;
genvar j;

generate
    for (i = 0; i < SIDE; i = i + 2) begin :gen_pool_row
        for (j = 0; j < SIDE; j = j + 2) begin :gen_pool_col
            assign imagem_out[((SIDE * i) / 4) + (j / 2)] = `MAX2(
                `MAX2(imagem_in[SIDE * (i) + j], imagem_in[SIDE * (i) + j + 1]),
                `MAX2(imagem_in[SIDE * (i + 1) + j], imagem_in[SIDE * (i + 1) + j + 1])
            );
        end
    end
endgenerate

endmodule
```

O bloco generate (talvez devesse ser só um for...) atribui à imagem de saída apenas o valor máximo a cada 4 valores de entrada dentro de um quadrado 2x2 na imagem. A imagem de entrada deve ser SEMPRE NxN tal que N é par (uma assertion).



Bloco ReLU: Função de ativação que transforma valores negativos em 0 e mantém valores positivos.

```
module ReLU #(
    parameter DATA_WIDTH = 32
) (
    input logic signed [DATA_WIDTH-1:0] in,
    output logic signed [DATA_WIDTH-1:0] out
);

localparam MSB = DATA_WIDTH - 1;

// Saída assume 0 para entrada negativa, e assume a própria entrada, para entrada positiva.
// Checa número negativo analisando o último bit: em complemento de 2, último bit 1 significa nº negativo.
assign out = in[MSB] ? '0 : in;

endmodule
```

Bloco quantizer: Quantizador, necessário para retornar ao ponto fixo de operação. Uma multiplicação de 4 bits com 4 bits tem como resultado 8 bits, mas suponha que todos os blocos trabalham com valores de 4 bits. O quantizador tem essa tarefa: de voltar o valor aos 4 bits perdendo o mínimo de informação possível. Como trabalhamos em ponto fixo, estamos descartando partes fracionárias.

```

module Quantizer #(
    parameter SIZE_IN = 16, //tamanho do barramento de entrada (geralmente vem de multiplicações e somas acumuladas)
    parameter SIZE_OUT = 8, //tamanho do barramento de saída
)
/*
| O shift é sempre válido somente se as escalas forem potências de 2, pois um shift divide/multiplica por 2.
| Felizmente, as escalas fracionárias utilizadas em ponto fixo são sempre potências de 2.
| O shift serve para REALINHAR a escala fracionária escolhida para as camadas trabalharem.
| Ex: inputs em Q1.7 e weights em Q1.7. A multiplicação entre eles vai gerar um Q2.14. A escala que a outra camada espera também é Q1.7
| Logo, espera-se que o shift elimine 7 bits fracionários da escala resultante para voltar à escala desejada. Então a conta do shift é
| num.de.bits.frac.input + num.de.bits.frac.weights - num.de.bits.frac.output.
| Isso gera Q2.7 (9 bits). Como o registro de saída só reserva SIZE_OUT bits (formato Q1.7, 8 bits), qualquer valor cujo resultado real precise
| do segundo bit inteiro será saturado no máximo/mínimo de Q1.7.
*/
parameter SHIFT = 1,
localparam signed [SIZE_IN-1:0] QMAX = (1 <<< (SIZE_OUT - 1)) - 1, // valor máximo positivo. Ex: 8 bits = 1 <<< 7 - 1 = 10000000 - 1 = 01111111
localparam signed [SIZE_IN-1:0] QMIN = -(1 <<< (SIZE_OUT - 1)) // valor mínimo negativo. Ex: 8 bits = 1 <<< 7 = 10000000
) (
    input logic signed [SIZE_IN-1:0] in,
    output logic signed [SIZE_OUT-1:0] out
);

logic signed [SIZE_IN-1:0] rounded;
// in + (1 com shift de SHIFT - 1) introduz "meio bit" na escala de saída (shift de SHIFT é igual a um shift de (SHIFT - 1) / 2), serve para não ocorrer arredondamento
// sempre para baixo.
assign rounded = (in + (1 <<< (SHIFT - 1))) >>> (SHIFT); // ERRO DE PARAMETRIZACAO: requer SIZE_IN > SIZE_OUT; com SHIFT <= 0, SHIFT-1 e deslocamentos negativos/inadequados.

// Introduz saturação para, caso o número shiftado para escala de saída dê maior que a máxima representação possível com o número de saída de bits,
// ou menor que a mínima representação, o resultado saturar.
assign out = ((rounded > QMAX) ? QMAX[SIZE_OUT-1:0] : ((rounded < QMIN) ? QMIN : rounded[SIZE_OUT-1:0]));
endmodule

```

Bloco Flatten: Pega as últimas featureMaps antes da camada de neurônios, e planifica-os. Isso significa torná-los unidimensionais. Apenas para compatibilidade com os neurônios, fazendo papel de entradas.

```
module Flatten #(
    parameter DATA_WIDTH = 8,
    parameter N_FEATURES = 1,
    parameter N_PIXELS_FEATURE = 10,

    localparam FLAT_OUT = N_PIXELS_FEATURE * N_FEATURES
) (
    input logic signed [DATA_WIDTH-1:0] features_in [0:N_FEATURES-1][0:N_PIXELS_FEATURE-1],
    output logic signed [DATA_WIDTH-1:0] flat_out [0:FLAT_OUT-1]
);

integer i;
integer j;

always @(*) begin
    for (i = 0; i < N_FEATURES; i = i + 1) begin
        for (j = 0; j < N_PIXELS_FEATURE; j = j + 1) begin
            flat_out[N_PIXELS_FEATURE * i + j] = features_in[i][j];
        end
    end
end

endmodule
```

Entram as features, e saem as mesmas, planificadas em uma dimensão.

Bloco CNN: Top module, que declara todos os outros blocos para formar a rede como um todo.

```
module CNN #(
    parameter DATA_WIDTH = 8, //largura do barramento de dados, utilizado em pesos e entradas
    parameter IMG_SIZE_1 = 32, //tamanho do lado da imagem
    parameter KERNEL_SIZE = 3, //tamanho do lado dos kernels
    parameter N_CHANNELS = 3, //número de canais iniciais da imagem
    parameter N_FILTERS_LAYER_1 = 2, //número de filtros da primeira camada
    parameter N_FILTERS_LAYER_2 = 4, //número de filtros da segunda camada
    parameter PADDING_1 = 1, //padding da primeira camada
    parameter PADDING_2 = 1, //padding da segunda camada
    parameter N_NEURONS_1 = 8, //número de neurônios da primeira camada Fully Connected
    parameter N_NEURONS_2 = 10, //número de neurônios da última camada (classes finais)
    parameter SHIFT_1 = 1, //shift do Quantizer (ver arquivo para mais detalhes). Deve vir de um script externo, que indica o formato de ponto fixo ad
    parameter SHIFT_2 = 1, //idem, para camada 2
    parameter SHIFT_3 = 1, //camada FC1
    parameter SHIFT_4 = 1, //camada FC2

    //Todas as etapas da CNN devem envolver tamanhos pares. Caso necessário, utilizar padding para transformar em par.
    localparam N_KERNEL = KERNEL_SIZE * KERNEL_SIZE, //número de elementos totais do kernel
    localparam N_PIXELS = IMG_SIZE_1 * IMG_SIZE_1, //número de pixels totais da imagem
    localparam OUT_SIZE_1 = IMG_SIZE_1 + 2 * PADDING_1 - KERNEL_SIZE + 1, //tamanho do lado das (pré) feature maps de saída, resultado do processament
    localparam N_OUT = OUT_SIZE_1 * OUT_SIZE_1, //número de elementos totais das (pré) feature maps de saída da primeira camada
    localparam N_OUT_POOLED = N_OUT / 4, //número de elementos totais das feature maps após a aplicação do maxPooling
    localparam IMG_SIZE_2 = OUT_SIZE_1 / 2, //tamanho do lado da "imagem" de entrada da segunda camada, que na verdade são as feature maps totalmente
    localparam N_CHANNELS_2 = N_FILTERS_LAYER_1, //número de canais da segunda camada, sendo equivalente ao número de filtros adotados pela primeira c
    localparam OUT_SIZE_2 = IMG_SIZE_2 + 2 * PADDING_2 - KERNEL_SIZE + 1, //tamanho do lado das (pré) feature maps de saída da segunda camada, resulta
    localparam N_OUT_2 = OUT_SIZE_2 * OUT_SIZE_2, //número de elementos totais das (pré) feature maps de saída da segunda camada
    localparam N_OUT_POOLED_2 = N_OUT_2 / 4, //número de elementos totais das feature maps da segunda camada após a aplicação do maxPooling
    localparam N_FLAT = N_OUT_POOLED_2 * N_FILTERS_LAYER_2 //número de elementos totais de todos os filtros da segunda camada somados, gerando uma cam
) (
    input logic clk,
    input logic rst,
    input logic start,
    input logic signed [DATA_WIDTH-1:0] imagem [0:N_CHANNELS-1][0:N_PIXELS-1],
    input logic signed [DATA_WIDTH-1:0] kernel1 [0:N_FILTERS_LAYER_1-1][0:N_CHANNELS-1][0:N_KERNEL-1],
    input logic signed [DATA_WIDTH-1:0] kernel2 [0:N_FILTERS_LAYER_2-1][0:N_FILTERS_LAYER_1-1][0:N_KERNEL-1],
    input logic signed [DATA_WIDTH-1:0] FCWeight1 [0:N_NEURONS_1-1][0:N_FLAT-1],
    input logic signed [DATA_WIDTH-1:0] FCWeight2 [0:N_NEURONS_2-1][0:N_NEURONS_1-1],
    input logic signed [DATA_WIDTH-1:0] ConvBias1 [0:N_FILTERS_LAYER_1-1],
    input logic signed [DATA_WIDTH-1:0] ConvBias2 [0:N_FILTERS_LAYER_2-1],
    input logic signed [DATA_WIDTH-1:0] FCBias1 [0:N_NEURONS_1-1],
    input logic signed [DATA_WIDTH-1:0] FCBias2 [0:N_NEURONS_2-1],
    output logic signed [DATA_WIDTH-1:0] result_FC2 [0:N_NEURONS_2-1],
    output logic valid_out4
);
```

Entradas: imagem – a imagem inicial. Kernel1 – os pesos que formam o primeiro kernel. Kernel2 – os pesos que formam o segundo kernel. FCWeight1 e 2 – os pesos que formam a primeira e segunda camada de neurônios. ConvBias1 e 2 – Os bias das camadas convolucionais. FCBias1 e 2 – Os bias das camadas de neurônios.

Saídas: Result\_FC2 – Resultado final, que determina a predição de classe – se conseguíssemos fazer funcionar a CNN no tamanho normal. E valid\_out4, que é o valid\_out que determina que a saída final está pronta para ser lida.

